

File Read Functions

[< Back to File Open Modes](#) | [Forward to The Input Files >](#)

C provides four functions that can be used to read files from disk:

1. [fscanf\(\)](#)
2. [fgets\(\)](#)
3. [fgetc\(\)](#)
4. [fread\(\)](#)

[< Back to File Open Modes](#) | [^ Up to Top](#) | [Forward to The Input Files >](#)

fscanf()

fscanf() is a *field* oriented function and is *inappropriate* for use in a robust, general-purpose text file reader. It has two major drawbacks:

- You must know the exact data layout of the input file in advance and rewrite the function call for every different layout.
- It's difficult to read text strings that contain spaces because fscanf() sees space characters as field delimiters.

For these reasons, I chose *not* to include fscanf() in this presentation. The [program](#) included here *does*, however, have the ability to read text files using fscanf(). If you [download](#) and compile it, you can demonstrate the limitations for yourself.

[< Back to File Open Modes](#) | [^ Up to Top](#) | [Forward to The Input Files >](#)

fgets()

fgets() is a *line* oriented function. The ANSI prototype is:

```
char *fgets(char *s,  
            int n,  
            FILE *stream);
```

The function reads from the stream pointed to by **stream** and places the output into the character array pointed to by **s**. It will stop reading when any of the following conditions are true:

- It has read **n - 1** bytes (one character is reserved for the null-terminator), or
- It encounters a *newline* character (a line-feed in the compilers tested here), or
- It reaches the end of file, or
- A read error occurs.

`fgets()` automatically appends a null-terminator to the data read.

On the surface, `fgets()` seems like the ideal function to read a text file line-by-line, but as the [program output](#) shows, it is not without limitations.

[< Back to File Open Modes](#) | [^ Up to Top](#) | [Forward to The Input Files >](#)

`fgetc()`

`fgetc()` is a *character* oriented function. The ANSI prototype is:

```
int fgetc(FILE *stream);
```

The function returns the *next* character in the stream pointed to by **stream** as an **unsigned char** converted to an **int**. It will stop reading at the end of file or if a read error occurs.

`fgetc()` gives you more control than `fgets()`, but reading a file byte-by-byte from disk is rather inefficient.

[< Back to File Open Modes](#) | [^ Up to Top](#) | [Forward to The Input Files >](#)

`fread()`

`fread()` is a *block* oriented function. The ANSI prototype is:

```
size_t fread(void *ptr,
             size_t size,
             size_t nmemb,
             FILE *stream);
```

The function reads from the stream pointed to by **stream** and places the output into the array pointed to by **ptr**. It will stop reading when any of the following conditions are true:

- It has read **nmemb** elements of size **size**, or
- It reaches the end of file, or
- A read error occurs.

`fread()` gives you as much control as `fgetc()`, and has the advantage of being able to read more than one character in a single I/O operation. In fact, memory permitting, you can read the *entire file* into an array and do all of your processing in memory. This has significant performance advantages.

`fread()` is often used to read fixed-length data records directly into **structs**, but you can use it to read *any* file. It's my personal choice for reading most disk files.

The [program](#) included here demonstrates how to read an entire file into an array using `fread()`. It also contains an example of *dynamic memory allocation* to resize an array so that it can accommodate the whole file without knowing the file's size in advance.

[< Back to File Open Modes](#) | [^ Up to Top](#) | [Forward to The Input Files >](#)
